

🧠 Topic 1: Process Management

Q What is a Process?

A process is an instance of a program in execution.

Imagine a program is like a recipe, and a process is like someone actually cooking using that recipe.

🧩 Components of a Process:

A process contains:

- Program code (text section)
- Program counter (PC) — next instruction to execute
- Stack — stores temporary data (Function parameters, return address)
- Heap — dynamic memory (for variables created during execution)
- Data section — global variables

📁 Process Control Block (PCB)

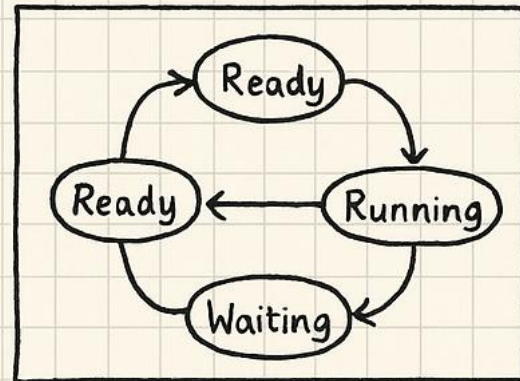
Think of this as the *identity card* or profile of a process. It contains:

- Process ID (PID)
- Process state
- Program counter
- CPU registers
- Scheduling info
- Memory Hack: Use the acronym

SPERMSI

State, PC CPU
regs Resources.
Memory Schedu.
ling, I/O

State, PC, CPU regs,
Resources, Memory,
Scheduling, I/O



🔄 State Transition Diagram:

State	Description
New	Process is being created
Ready	Waiting to be assigned CPU
Running	Instructions are being executed
Waiting	Waiting for an event like I/O
Terminated	Finished execution

📁 Process Control Block (PCB)

Think of this as the *identity card* or profile of a process. It contains:

- Process ID (PID)
- Process state
- Program counter
- CPU registers
- Scheduling info
- Memory limits
- I/O info

🔄 Context Switching

When the CPU switches from one process to another, it:

- Saves current process's state in PCB
- Loads the new process's state

This is called a context switch, and it has a cost (time overhead).



Topic 2: Process Synchronization

We'll cover:

- Race Condition
- Critical Section
- Why Synchronization is Needed



What is a Race Condition?

Race condition occurs when two or more processes access shared data at the same time, and the outcome depends on the timing of their execution.

⚠ Memory hack: "Race condition = Racing to access shared data = Unpredictable results"



Example (Real-Life Style):

The Critical section editing the same Google Doc without accesses shared resources (like variables, files, printers).

🔒 Only one process should execute its critical section at a time to prevent race conditions.

~~Imagine a single bathroom with many people (processes) wanting to use it at the same time - disaster!~~
We need a lock on the door - synchronization mechanism



Rules to Handle Critical Section (The 3 Conditions)

These 3 are essential conditions for any synchronization:

Condition	Meaning	Meaning
Mutual Exclusion	Only one process can enter critical	Only one process can enter critical section at a time
Progress	If no process is in CS, decision of who enters next. is not postponed	
Bounded Waiting	A process must get a chance to enter CS within limited time.	



Mnemonic: MPB = Most Precious Bathroom



Why Synchronization?

Without synchronization:

- You get race conditions

Topic 2: Process Synchronization

Why Synchronization:

- ⚠️ You get race conditions
- 🔥 Data corruption
- ✖ Deadlocks

1. What is a Race Condition?

Race condition occurs when two or more processes access shared data at the same, and the outcome depends on the timing of their execution.

- ⚠️ Memory hack: "Racing condition = Racing to access shared data = Unpredictable results"

✖ Rules to Handle Critical Section (The 3 Conditions)

These 3 are essential conditions for any synchronization solution:

Condition	Meaning
Mutual Exclusion	Only one process can enter critical section
Progress	If no process is in CS, decision of who enters next is not postponed
Bounded Waiting	A process must get a chance to enter CS within limited time

Why is it important?

You get race conditions
Data corruption
Deadlocks

We'll dive deeper into each of these in the next topics!



Topic 3: Peterson's Solution

It's a software-based solution for mutual exclusion between two processes only.

Basic Idea:

It allows two processes to take turns entering the critical section without interfering with each other



Memory hack:

"Race condition = Racing to access shared data = Unpredictable results"



Shared Variables Used:

boolean flag[2]: // indicates if a process wants to enter CS



Peterson's Algorithm (for Process 0):

```
flag(0) = true;
turn = 1
```

```
while (flag(1) == true
      && turn == 1);
// Critical Section
```

```
flag(0) = false;
```

→ ~~Remember: IDB = 100~~
~~Peterson's Solution~~



Conditions Met:

Condition	Met?
Reason	Reason

Peterson's Algorithm (for Process 0):

```
flag(0) = true;
turn = 1
```



```
while (flag(1) == true
      && turn == 1):
    // Critical Section
```

```
flag(0) = false;
```



Step-by-Step Visualization:

- You get race conditions
- Data corruption
- Deadlocks



Why Synchronization?

Without synchronization:

- You get race condition
- Gives turn to 1
- They don't wait unnecessarily + it's their turn



Memory Trick:

F-T-W → Flag, Turn, Wait
Every Peterson process follows:

```
Flag = true
Turn = other
```

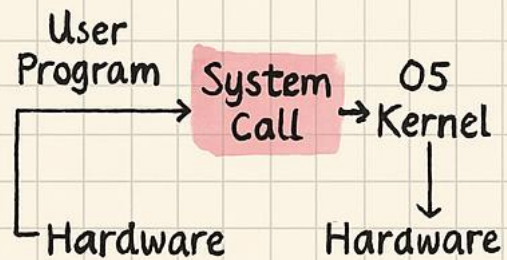

Topic 4: System Calls & Their Types

📞 What is a System Call?

A System Call is how a User-level program requests a service from the Operating System (OS) – like talking to the OS.

🧠 Real-life analogy:

you (a program) call room service (system call) to bring food (resource) - you can't enter the kitchen (kernel), so you request.

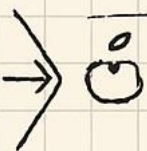


🏠 Where System Calls Belong:

They are the interface between user programs and the OS kernel.

👑 Where System Calls Belong: They can:

They are interface between user programs and the OS kernel.



Allocate memory
Allocate memory
Run a new program
Run a new program

📁 Types of System Calls Mnemonic: F-DC-PIM

These 3 are essential conditions for any synchronization solution.

Type	Purpose	Examples
1. File Management	Create, delete, read,	open(), read(), write()
2. Device Control	Request, release, attach	fork(), exit(), exec()
3. Process Control	Create, terminate	getpid(), alarm(), time()
4. Information Management	Get/set system data,	pipe(), shmget(), msgget()
5. Communication	user into between processes (IPC)	pipe(), shmget(), msgsnd()
6. Memory Management	Allocate and free memory	malloc(), free() indirect)

🔗 Why Synchronization?

Without synchronization:

- You get race conditions
- Data corruption

File Devices ⇒ Can Provide Important Memory
(Just remember: FDC-PIM)



Topic 5: Types of Operating Systems

Operating systems aren't just Windows or Linux – there are different types based on how they handle users, tasks, and resources.

We'll cover:

Types of Operating Systems



1. Batch Operating System

- No interaction with user during execution
- Jobs with similar needs are batched together
- Uses Job Control Language (JCL)
- Oldest type of OS



Example: Early IBM mainframes



2. Time-Sharing OS (Multitasking)

- Multiple users share system simultaneously
- CPU switches rapidly (context switching)
- Response time is short



3. Real-Time OS

- Gives immediate response
- Used in embedded systems, robotics, aircraft



Example: VxWorks, QNX



6. Mobile OS

- Designed for smartphones/tablets
- Touch support, app stores, power-saving



Example: Android, iOS



9. Multiprocessing OS

- Uses more than one processor (CPU) to execute tasks
- Increases speed and reliability



3. Distributed OS

- Combines many computers to act like one
- Tasks are distributed among multiple machines
- Appears like a single system to the user



Think: "Divide to Conquer"



6. Mobile OS

- Designed for smartphone/tablets
- Touch support, app stores, power-saving



Example: (UNIX fold)

Memory Trick: "Program ke saath aur bhi programs"



9. Multitasking OS

- Multiple tasks executed by rapid switching (context switch)
- Makes system fast and responsive



Example: Windows, Linux

Summary Table:

OS Type	Key Feature	Example
Batch	Executes jobs in groups	IBM Mainframes
Time Sharing		Unix, Linux
Distributed Network	Many computers act as one	Amoeba, LOCUS, Windows Server
Real-Time	Instant response	VxWorks, FreeRTOS
Mobile	Optimized for	Android, iOS
Multiprogramming	Many programs in memory	Early Unix